

microcontroller $\rightarrow$ piece of silicon $\rightarrow$ when power hits it, the CPU cores wakes up and asks one question $\Rightarrow$ Where do I find my first Instruction?

That's it that is entire boot system!

The CPU does not know whether its OS Blink LED or nothing

- 1) It just need an Address to start executing from
- 2) Boot Process - chain of decision and code that leads CPU from ^{ee} just powered on to running application

STM32 F103C8 + After Power on

Step 1 $\rightarrow$ The moment reset released (power-on reset button watchdog reset any Reset)
CPU Hardware does two things automatically in silicon before any code runs $\rightarrow$

- 1) It Reads Value at Add $\rightarrow 0x00000000$
 $\downarrow$
- 2) It Reads Value at Add loads into SP
 $0x00000004 \leftarrow$
 $\downarrow$
loads into Program Counter
(PC)

3) Then it starts executing wherever PC
is pointing That's it that All
CPU Case About! 2 memory Reads
execute

Step 2] $\Rightarrow$ The BOOT Pins Decide What
Address $\rightarrow 0x00000000$ Actually IS

STM32 - Has Remapping Mechanism

- 1) Physical Memory are
- 1) Flash $\rightarrow$ Physically lives At $0x08000000$
 - 2) System Mem $\rightarrow 0x1FFFF000$ (factory boot loader)
 - 3) ~~BB~~ Embedded SRAM $\rightarrow 0x20000000$

But the CPU Reads its reset Vector
from 0x0000 0000 so what lives At

0x0000 0000

BOOT pin Has Answer before CPU
even Starts!

Boot1 = X, Boot0 = 0 $\rightarrow$ 0x0000 0000 is ALIASED to Flash
(0x0800 0000)

Boot1 = 0, Boot0 = 1 $\rightarrow$ 0x0000 0000 is $\rightarrow$ Syst memory
0x1FFF 1000

Boot1 = 1, Boot0 = 1 $\rightarrow$ 0x0000 0000 $\rightarrow$ SRAM $\rightarrow$ 0x2000 0000

Aliased Means Same physical memory is
Accessible at two Addresses Simultaneously

My Flash is at 0x0800 0000 and
0x0000 0000 At the Same time

So BOOT pins don't run any code

They just tell Address Decoder Hardware
which zoom to point to when someone
knows At 0x0000 0000

By Default Blue Pill Has Boot0 = 0 GND
which Means Flash Alias to 0x00000000

My Code Sits at flash Starting 0x08000000

The CPU boots read reset vector from 0x00000000. which is your Flash it finds SP and PC values & compiler placed there and jumps straight to your main

BOOT Pins (BluePill) Relevant when

- 1) want to flash firmware without ST-link (UART, USB, CAN - The Factory System memory bootloader handles this)
- 2) flash firmware is bricked and need to RECOVER
- 3) writing your own bootloader that lives in Flash and then jumps to Application

3 Boot Configuration → What Happens in Each

Config 1 → Boot from Flash (Normal)

Boot 0 = 0 Boot 1 = Anything

Power-ON → CPU reads $0x00000000$ → Flash $0x08000000$

→ $0x00000000$ ← reads SP e.g. → $0x20005000$
→ $0x00000000$ ← reads PC → $0x08000141$

↓ CPU jumps to ⇒ $0x08000141$

↓
RST Handler () runs ← This is startup code

Your toolchain
write

↓
- Copies .data Section from
Flash to
SRAM

↓
- zeroes out .bss
Section in SRAM

↓
- calls SystemInit() (Setup clock)

↓
main code runs

Configuration 2: Boot from System Memory

Boot 0 = 1 Boot 1 = 0

ST put Read-only, factory-programmed bootloader in System Memory. You

Cannot erase it. It speaks UART /
USB
CAN

Poweron → CPU Reads
0x00000000

Aliased to 0xFFFF0000



Factory bootloader runs

- Initialization. UART at auto-baud
- waits for Host tool (ZOE)
- RX my .bin/.hex file over UART
- writes it to Flash starting at 0x0800 0000

Reset with Boot 0 = 0 → Normal boot -
my code runs

This is how you flash Blue Pill using
UART-USB conn → (p2102)

Configuration 3 $\rightarrow$ Boot from SRAM

Boot0=1 Boot1=1

Code Runs from ~~Flash~~ RAM

for quick Iteration $\rightarrow$ code runs from RAM

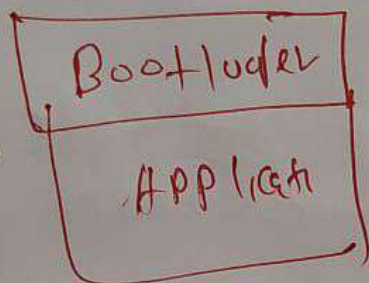
So you don't cycle Flash,

Flash layout with Custom Bootloader

0x08000000

0x08004000

0x0800FFFF



$\sim 8-16KB$

rest of Flash

CPU Always Boots from 0x08000000

So bootloader runs first Always. It then decides whether to jump to the Application or wait for new firmware

Bootloader Job in Pseudocode

1) Initialize min H/W (clock, UART, USB)

2) check : Should I enter Update mode
Is there is no App in flash

3) If YES $\rightarrow$ Rx new firmware, write to

0x08004000

4) Not Jump to 0x08004000